

Incident Response Playbook — PHI Data Breach via Chat IDOR (AS-01)

1. Detection

Triggers

- Security alert from `alerts.log` indicating unusual chat room access patterns
- Security log showing `ACCESS_DENIED` events on `/api/chat/history/room/` from unauthorized users
- Patient report of unauthorized access to their medical conversations
- Anomalous spike in chat history API calls from a single user accessing multiple rooms

Detection Sources

Source	Log File	Key Fields
Security Logger	<code>logs/security.log</code>	<code>AUTH_LOGIN_SUCCESS</code> , <code>SENSITIVE_ACCESS</code>
Alert System	<code>logs/alerts.log</code>	<code>BRUTE_FORCE_DETECTED</code>
Application Log	<code>stdout</code>	Chat room access patterns
Database Audit	PostgreSQL logs	Query patterns on <code>chat_messages</code> table

Sample Detection Query

```
# Find users accessing rooms they don't belong to
grep "ACCESS_DENIED" logs/security.log | grep "chat" | sort | uniq -c | sort -rn
```

2. Triage

Severity Assessment Matrix

Factor	Criteria	Score
Data Type	PHI (medical conversations, symptoms, SSN)	Critical

Factor	Criteria	Score
Volume	Each room contains full conversation history	High
User Impact	Patient medical privacy violated	Critical
Regulatory	HIPAA breach notification required if >500 records	Critical
Exploitability	Low-skill attack (URL parameter manipulation)	High

Triage Decision

- **P1 (Critical):** PHI confirmed accessed by unauthorized party → Activate full IR
- **P2 (High):** Suspicious access patterns but no confirmed PHI exposure → Investigate
- **P3 (Medium):** Failed access attempts only (blocked by controls) → Log and monitor

Immediate Questions

1. How many rooms were accessed by the unauthorized user?
2. What PHI was contained in those conversations?
3. How long has the vulnerability been exploitable?
4. Are there other users who exploited this?

3. Containment

Immediate Actions (< 15 minutes)

1. Revoke attacker's session:

```
-- Identify the attacker's user ID from security logs
-- Force re-authentication by rotating JWT secret
UPDATE users SET password_hash = 'LOCKED' WHERE id = <attacker_id>;
```

2. Deploy emergency fix:

```
# The ownership check fix (already implemented as AS-01 remediation)
# Verify fix is deployed:
grep "userId !== idA" backend/src/controllers/chatController.js
```

3. Restart application to clear any cached sessions

Short-Term (< 1 hour)

4. Audit all chat room access in the last 30 days:

```
SELECT DISTINCT sender_id, receiver_id, room_id,
               MIN(timestamp) as first_access, MAX(timestamp) as last_access
FROM chat_messages
GROUP BY sender_id, receiver_id, room_id;
```

- 5. Cross-reference with security logs to identify unauthorized access patterns
- 6. Disable public registration temporarily if attacker created accounts for exploitation

4. Investigation

Evidence Collection

Evidence	Source	Collection Method
Security logs	logs/security.log	cp logs/security.log evidence/security_\$(date +%s).log
Alert logs	logs/alerts.log	cp logs/alerts.log evidence/alerts_\$(date +%s).log
Database records	PostgreSQL	pg_dump -t chat_messages mediconnect > evidence/chat_\$(date +%s).sql
Application logs	stdout/stderr	Container log export
Git history	Repository	git log --since="30 days ago" -- src/controllers/chatController.js

Investigation Queries

```

-- Identify all unique room access patterns
SELECT room_id,
       COUNT(DISTINCT sender_id) as unique_senders,
       COUNT(*) as total_messages
FROM chat_messages
GROUP BY room_id
ORDER BY unique_senders DESC;

-- Find messages accessed by users not in the room
-- (After the fix, this would show historical unauthorized access)
SELECT cm.room_id, cm.sender_id, cm.receiver_id, cm.timestamp
FROM chat_messages cm
WHERE NOT EXISTS (
  SELECT 1 FROM users u
  WHERE u.id IN (
    CAST(SPLIT_PART(cm.room_id, '-', 1) AS INT),
    CAST(SPLIT_PART(cm.room_id, '-', 2) AS INT)
  ) AND u.id = cm.sender_id
);

```

Timeline Reconstruction

1. When was the vulnerable code deployed?
2. When was the first unauthorized access?
3. What rooms/PHI were accessed?
4. When was the vulnerability detected?
5. When was containment achieved?

5. Evidence Preservation

Chain of Custody

1. All evidence files must be:
 - Copied (not moved) to `evidence/` directory
 - SHA-256 hashed: `shasum -a 256 evidence/* > evidence/checksums.txt`
 - Timestamped with collection time
 - Stored in read-only state: `chmod 444 evidence/*`
2. Database snapshots:

```

pg_dump mediconnect > evidence/db_snapshot_$(date +%Y%m%d_%H%M%S).sql
shasum -a 256 evidence/db_snapshot_*.sql >> evidence/checksums.txt

```

3. Log preservation:

```
tar czf evidence/logs_$(date +%Y%m%d).tar.gz logs/
```

6. Impact Assessment

HIPAA Breach Risk Assessment (per §164.402)

Factor	Assessment
Nature of PHI	Medical symptoms, doctor-patient conversations, potential SSN
Unauthorized person	Other registered patient (known identity)
PHI actually viewed	Confirmed via security logs
Risk mitigated	Vulnerability patched, attacker access revoked

Notification Requirements (§164.404-408)

Threshold	Requirement	Timeline
< 500 individuals	Notify affected individuals + HHS annual log	Within 60 days
≥ 500 individuals	Notify individuals + HHS + media	Within 60 days

Patient Impact

- Medical conversation confidentiality breached
- Potential identity theft if SSN/government IDs were in messages
- Trust relationship with healthcare provider damaged

7. Remediation

Code Fix (Already Implemented)

- **AS-01:** Added room ownership verification in chatController.js
- **Verification:** Retest confirms Patient B receives 403 Forbidden when accessing Patient A's room

Infrastructure Hardening

- Rate limiting on chat history endpoint
- Additional logging for chat access patterns
- Regression test in CI/CD pipeline to prevent reintroduction

Process Improvements

- Mandatory security review for all endpoints handling PHI
- IDOR testing checklist for new API development
- Quarterly penetration testing schedule

8. Recovery

1. Confirm all unauthorized sessions are terminated
2. Verify fix deployment across all environments
3. Run regression tests to confirm fix effectiveness
4. Resume normal operations
5. Monitor security logs for 72 hours post-incident for related activity

9. Communication

Internal Notification

Stakeholder	Method	Content	Timeline
Engineering Lead	Slack/Email	Technical details + fix status	Immediately
CISO/Security	Formal report	Full IR report	Within 4 hours
Legal/Compliance	Meeting	Breach assessment + notification requirements	Within 24 hours
Executive Team	Summary	Business impact + remediation status	Within 48 hours

External Notification (if required)

Party	Method	Content	Timeline
Affected Patients	Written notice	What happened, what PHI, what we're doing	Within 60 days

Party	Method	Content	Timeline
HHS OCR	Breach portal	Formal breach notification	Within 60 days
State AG (if applicable)	Per state law	Breach notification	Per state requirement

10. Lessons Learned

Root Cause

Missing authorization check (IDOR) on chat history endpoint — the endpoint accepted any `roomId` without verifying the requesting user was a participant.

Contributing Factors

- No security review process for new endpoints
- No automated IDOR detection in CI/CD
- No monitoring for cross-user data access patterns

Preventive Measures

Measure	Owner	Deadline
Add IDOR regression tests to CI/CD	Security Engineering	Completed
Implement endpoint-level access logging	Backend Team	Completed
Create security review checklist for PRs	Security Team	1 week
Schedule quarterly penetration testing	Security Team	Ongoing
Add rate limiting to sensitive endpoints	Backend Team	2 weeks

Metrics to Track

- Time to detection
- Time to containment
- Number of affected records
- Recurrence rate of similar vulnerabilities